

Deep research “ jemalloc link-by-SONAME fix (install.sh exit 77)

Revision: 1 **Last modified:** 2026-06-30T00:00:00Z **Scope:** Â§11.4.150
mandatory deep multi-angle research for the build_native.sh jemalloc-link
fix (host path) + its zig-path exception.

Problem (FACT, captured)

bash scripts/install.sh on a base ALT host exited 77. Forensic chain:
rootless podman lacked newuidmap “ containerized build failed “
Â§11.4.101 native fallback ran build_native.sh “ ./configure died C
compiler cannot create executables. config.log real cause:

```
/usr/bin/ld.bfd: cannot find -ljemalloc: No such file or directory
```

The host ships a **runtime-only** jemalloc “ /lib64/libjemalloc.so.2 (+
.so.2.0) with **no** -devel libjemalloc.so symlink and no .a.
build_native.sh injected bare -ljemalloc into the configure-time LDFLAGS,
which the GNU linker cannot resolve without a libjemalloc.so symlink “
poisoning even the trivial “œcan the compiler link?“œ probe.

Captured proof (qa-results/loop-20260630/jemalloc-link-soname/): -
C1_bare_ljemalloc.log: /usr/bin/ld.bfd: cannot find -ljemalloc
(FAILS) - C1_soname_link.log: empty “ -l:libjemalloc.so.2 LINKS

Angle 1 “ GNU ld -l:namespec (authoritative)

GNU Binutils ld manual, *Options* (-l namespec / --library=namespec):

“œIf namespec is of the form :filename, ld will search the
library path for a file called filename “| which always specifies a
file called filename.“œ

“ -l:libjemalloc.so.2 resolves the **exact** runtime SONAME file with no
name-mangling and **no dev symlink required**. This is the correct, root-free
resolution for a runtime-only host jemalloc, and aligns with the project“™s
Â§11.4.111 resolve-by-stable-name discipline (the resolved JEMALLOC_SO
SONAME basename is the stable identifier).

Source: <https://sourceware.org/binutils/docs/ld/Options.html> (verified
2026-06-30).

Angle 2 “zig cc does NOT support -l: (authoritative + empirical)

Applying the same -l: form to the **zig** native-build path (the ROOT-FREE obtained-toolchain branch) BROKE it: configure died C compiler cannot create executables under the obtained zig toolchain.

Authoritative confirmation “ziglang/zig issue #10851: “zig cc doesn’t handle -l :\$FILE” underlying ld.lld: error: cannot open :libfoo.so.1. -l: is a GNU-ld extension zig cc/LLD does not accept.

Source: <https://github.com/ziglang/zig/issues/10851> (verified 2026-06-30).

Empirical corroboration: scripts/tests/71_root_free_zig_build.sh PASSED (12/0/0) on the pre-fix tree (bare -ljemalloc), FAILED with -l: applied, and PASSED again (12/0/0) after the zig line was reverted to bare “ isolated in a throwaway git worktree (11.4.114). The zig path’s jemalloc is a LOCAL source-build whose make install ships a libjemalloc.so dev symlink, so bare -ljemalloc (+ -L\$PFX/lib) resolves correctly there.

Resolution (two-mechanism, 11.4.111 honest boundary)

Path	jemalloc provenance	dev symlink?	linker	link token
host-toolchain (LDFLAGS)	host runtime-only .so.2	NO	GNU ld	-l:\$(basename "\$JEMALLOC_SO") ({JEM_LINK})
zig (ZLDFLAGS)	local source-build	YES	ld.lld	bare -ljemalloc
macOS (LDFLAGS)	brew	YES (.dylib)	ld64	bare -ljemalloc

Confirm-no-bigger-problem (11.4.150(C))

The fix is a pure **link-token** change. The produced binary’s DT_NEEDED is unchanged “ ldd tmux/build/bin/tmux shows libjemalloc.so.2 => /lib64/libjemalloc.so.2 (same SONAME recorded as before), and tmux -V = tmux 3.6a. No runtime behaviour change; only the configure/link-time resolution is fixed. No deeper defect masked: the original failure was a missing dev symlink, not a broken jemalloc.

Sources verified 2026-06-30

- GNU Binutils ld manual “ Options (-l:namespec): <https://sourceware.org/binutils/docs/ld/Options.html>

- ziglang/zig#10851 (zig cc no -l: support): <https://github.com/ziglang/zig/issues/10851>
- Captured evidence: qa-results/loop-20260630/jemalloc-link-soname/C1_*.log, test 71 isolation (pre-fix PASS / -l: FAIL / reverted PASS).